



8.1 开发设计规范



开发设计规范概述

1.数据建模

- (1) 规范化：确保表结构设计合理，避免数据冗余和更新异常。
- (2) 非规范化：在某些场景下，适度的非规范化可以提高查询性能，但需谨慎使用。
- (3) 索引设计：合理设计索引以加速查询，同时注意索引对写入性能的影响。



开发设计规范概述

2.SQL 编写规范

- (1) SQL 语句优化：使用高效的查询语句，如避免SELECT *, 尽量使用 EXPLAIN分析 SQL 执行计划。
- (2) 事务管理：正确使用事务来保证数据一致性。
- (3) 存储过程：当逻辑复杂时，考虑使用存储过程减少网络传输开销。
- (4) 函数与触发器：合理利用用户定义的函数和触发器实现复杂的业务逻辑。



开发设计规范概述

3.应用程序设计

- (1) 连接池：使用连接池管理数据库连接，减少频繁创建和销毁连接带来的性能损耗。
- (2) 错误处理：应用程序需要能够妥善处理数据库操作中的各种错误情况。
- (3) 并发控制：确保多线程或多进程环境下的数据一致性，防止死锁等问题。
- (4) 安全性：确保数据库的安全性，比如使用参数化查询防止SQL注入攻击。



开发设计规范概述

4.性能调优

- (1) 监控工具：使用openGauss 提供的监控工具监控数据库性能。
- (2) 配置优化：根据实际应用场景调整数据库配置参数。
- (3) 定期维护：定期执行 VACUUM 和 ANALYZE 操作来整理表空间和更新统计信息。



开发设计规范概述

5.开发流程

- (1) 版本控制：使用版本控制系统管理代码变更。
- (2) 自动化测试：编写单元测试和集成测试，确保应用的稳定性。
- (3) 持续集成/持续部署 (CI/CD)：采用 CI/CD 流程自动化构建、测试和部署过程。

6.文档与注释

- (1) 文档编写：编写清晰的系统文档，包括数据库设计文档、接口文档等。
- (2) 代码注释：为关键部分添加必要的注释，便于后期维护。



数据库对象命名

- (1) 避免使用保留或者非保留关键字命名数据库对象。
- (2) 避免使用双引号括起来的字符串来定义数据库对象名称，除非需要限制数据库对象名称的大小写，数据库对象名称大小写敏感会使定位问题难度增加。
- (3) 数据库对象命名风格务必保持一致。数据库对象名称尽量做到望文知意，避免使用自定义缩写（可以使用通用的术语缩写）。例如，在命名中可以使用具有实际业务含义的英文词汇或汉语拼音，但规则应该在数据库实例范围内保持一致。变量名的关键是要具有描述性，即变量名称要有一定的意义，还应有前缀标明该变量的类型。
- (4) 表对象的命名应该可以表征该表的重要特征。例如，在表对象命名时区分该表是普通表、临时表还是非日志表。



数据库对象设计

1.Database和Schema设计

在openGauss中，可以使用Database和Schema实现业务的隔离，区别在于Database的隔离更加彻底，各个Database之间共享资源极少，可实现连接隔离、权限隔离等，Database之间无法直接互访。Schema隔离的方式共用资源较多，可以通过grant与revoke语法便捷地控制不同用户对各Schema及其下属对象的权限。

(2) Schema设计建议

如果该用户不具有sysadmin权限，或者不是该Schema的owner，该用户访问Schema下的对象时，需要同时赋予用户Schema的usage权限和对象的相应权限。



数据库对象设计

1.Database和Schema设计

2.表设计

3.规划存储模型

4.字段设计

5.视图和关联表设计